

SCHOOL BUS TRACKING PORTAL

A Web-Based Child Safety, School Transport & Live Tracking System

PROJECT DOCUMENTATION

TEAM MEMBER	ROLL NUMBER
Moksha	2520030515
Gayatri	2520030600
Anshu	2520030440

Academic Project • Web Application • Child Safety & School Transport

Prepared as a structured software engineering project report

ACKNOWLEDGEMENT

We express our sincere gratitude to our faculty members and institution for providing the opportunity to design and develop the School Bus Tracking Portal. The project allowed us to apply concepts from web development, databases, software engineering, user interface design, and system testing to a practical child-safety problem.

We also acknowledge the value of collaborative development. The project required us to analyse user needs from the perspectives of parents, school administrators, drivers, and students, and to translate those needs into a structured digital solution.

This documentation presents the problem, objectives, system requirements, technology stack, architecture, design, implementation approach, features, testing, deployment considerations, limitations, future enhancements, and conclusion of the project.

ABSTRACT

The School Bus Tracking Portal is a web-based child-safety and school transportation management system designed to improve visibility, communication, and accountability during student travel. The system provides separate workflows for parents and administrators, allowing authorised users to view transport information, track buses, monitor boarding and alighting events, and receive alerts related to school transportation.

A central goal of the portal is to move beyond generalized stop information. Each route stop can be represented using a specific location, including meaningful stop names and geographic coordinates. The system can therefore communicate not only that a bus is approaching a stop, but also which location is relevant to the child. The child's scheduled alighting stop can be highlighted so that a parent can immediately identify the intended get-down point.

The portal is designed around a modular architecture consisting of a responsive frontend, an API/backend layer, structured transport data, and a location-tracking component. Core entities include students, parents, buses, drivers, routes, stops, trips, boarding records, alighting records, GPS updates, and alerts. The academic prototype emphasises a clear user interface and structured data model while allowing the live GPS and production database services to be integrated as deployment requirements mature.

The project demonstrates how software engineering principles can be applied to a safety-sensitive everyday problem. By combining transport records, location information, event tracking, and role-based dashboards, the proposed system aims to reduce uncertainty for parents and improve operational visibility for schools.

TABLE OF CONTENTS

Chapter 1	— Introduction
Chapter 2	— System Requirements
Chapter 3	— Technology Stack
Chapter 4	— System Architecture
Chapter 5	— System Design
Chapter 6	— Implementation
Chapter 7	— Features
Chapter 8	— Testing
Chapter 9	— Deployment
Chapter 10	— Challenges & Limitations
Chapter 11	— Future Enhancements
Chapter 12	— Conclusion
Chapter 13	— References
Chapter 14	— Appendices

Note: Page numbering is generated automatically in the final PDF.

CHAPTER 1

INTRODUCTION

1.1 Background of the Project

School transportation is a routine part of student life, but it also creates a period during which parents have limited visibility of a child's journey. Traditional transport systems may depend on verbal updates, fixed stop lists, or manual attendance records. These methods can make it difficult for parents to know whether a child has boarded, where the bus currently is, or whether the child has reached the intended alighting location.

The School Bus Tracking Portal addresses this gap through a central digital interface. Instead of treating a bus route as a simple list of generalized stops, the system models each stop as a meaningful location. This supports clearer route visualization and allows the portal to identify the child's specific get-down stop.

1.2 Problem Statement

Parents need timely and understandable information about a child's school-bus journey, while school administrators need reliable transport records and operational visibility. A fragmented process can lead to repeated calls, uncertainty about boarding or alighting, difficulty tracing trips, and limited visibility into bus movement.

The problem is therefore to design a secure, user-friendly portal that connects student transport records with route, stop, boarding, alighting, GPS, and alert information while keeping the parent experience simple.

1.3 Scope of the Project

- Parent-facing transport dashboard for student and trip information.
- Administrator-facing management of students, buses, drivers, routes, trips, boarding, alighting, GPS and alerts.
- Location-specific stop information with latitude/longitude support.
- Visual identification of the child's scheduled alighting stop.
- Structured data model suitable for REST API and database integration.
- Responsive web interface for desktop and smaller screens.
- Testing of major workflows using functional and API-oriented test cases.

1.4 Objectives

- Improve parent visibility into school-bus journeys.
- Represent route stops as meaningful, location-specific entities.
- Make the child's intended alighting point immediately identifiable.
- Maintain structured records for students, buses, routes and trips.
- Provide an administrator workflow for transport monitoring.
- Create a foundation that can be connected to live GPS and production database services.

1.5 Importance of the Application

The project is important because transport safety depends not only on knowing where a bus is, but also on knowing which student, route, stop, and trip are associated with that location. A digital record of boarding and

alighting can improve traceability, while alerts can provide an additional communication channel for unusual events or important trip milestones.

1.6 Target Users / Audience

Parents: view the child's transport information, bus status, route, relevant stop and safety-related alerts.

School Administrators: manage transport entities and monitor trips, attendance events, GPS information and alerts.

Drivers / Transport Staff: serve as operational actors whose trip, bus and route information is maintained by the system.

Students: are represented as transport records; the portal is primarily operated by authorised adults.

CHAPTER 2

SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Component	Recommended Requirement
Processor	Modern dual-core processor or better
RAM	4 GB minimum; 8 GB recommended
Storage	At least 2 GB free for project and dependencies
Display	1366×768 or higher for desktop testing
Network	Stable internet for development and map/location services

2.2 Software Requirements

- Windows/macOS/Linux development environment.
- Node.js and npm for frontend/backend development.
- Modern web browser such as Chrome, Edge or Firefox.
- Code editor such as Visual Studio Code.
- Database service suitable for the selected deployment architecture.
- Postman or equivalent tool for API validation.

2.3 Functional Requirements

ID	Requirement
FR-01	User authentication must provide authorised access to the appropriate dashboard.
FR-02	The system must maintain student and parent transport associations.
FR-03	The system must maintain buses, drivers, routes and stops.
FR-04	Each stop should support a specific location and coordinates.
FR-05	The parent view should identify the child's alighting stop.
FR-06	Trip/GPS information should be available to the tracking interface when connected.
FR-07	Boarding and alighting events should be recorded.
FR-08	Administrators should be able to review alerts and transport activity.

2.4 Non-Functional Requirements

- Usability: parent-facing screens should be understandable without technical knowledge.
- Performance: dashboard data should load promptly under normal network conditions.
- Security: only authorised users should access student and location information.

- Reliability: transport records should remain consistent across modules.
- Maintainability: components, routes, controllers and data entities should be modular.
- Responsiveness: the interface should adapt to common desktop and mobile viewport sizes.

2.5 User Roles

The portal uses role-oriented access. Parent users focus on the journey of an associated child, while administrators have broader transport-management visibility. This separation reduces unnecessary exposure of unrelated student information and keeps each dashboard focused on its intended task.

CHAPTER 3

TECHNOLOGY STACK

3.1 Front-End

The frontend is designed as a component-based web application. React.js is suitable for rendering dashboards, route views, cards, tables, alerts and interactive tracking components. CSS and responsive layout techniques are used to maintain a consistent visual hierarchy across screen sizes.

3.2 Back-End

The backend architecture is designed around Node.js and Express.js. It provides a natural REST API layer between the frontend and persistent data. Typical resources include students, buses, drivers, routes, stops, trips, boarding records, alighting records, GPS updates and alerts.

The current academic prototype should be distinguished from a production deployment: a frontend can demonstrate the complete user experience with mock data even before all live API and GPS services are connected. The architecture therefore keeps the API boundary explicit so that dynamic services can be integrated without redesigning the user interface.

3.3 Database

A relational database such as MySQL is appropriate for the structured transport records because entities such as students, parents, buses, routes, trips and boarding/alighting events have clear relationships. Location updates can be stored as timestamped latitude/longitude records associated with a trip or bus.

3.4 Location Services

The location layer uses geographic coordinates to represent specific stops and vehicle positions. A map component can render the current bus marker, route path and stop markers. Location updates should be timestamped so that the system can distinguish the latest known position from older records.

3.5 Version Control

Git and GitHub can be used to maintain project history, coordinate team changes and preserve stable versions. Feature branches, meaningful commit messages and a shared README improve collaboration and reproducibility.

3.6 API / External Services

- REST API between frontend and backend.
- Geolocation or map service for map rendering and coordinates, when enabled.
- Browser location capabilities or GPS device feed for live vehicle position, where supported.
- Optional notification service for future SMS, email or push alerts.

3.7 Technology Summary

Layer	Technology / Approach
UI	React.js, HTML, CSS, responsive components

Server	Node.js + Express.js REST API
Database	MySQL / structured relational schema
Location	GPS coordinates + map integration
Testing	Postman, browser-based functional testing
Development	Visual Studio Code, npm, Git

CHAPTER 4

SYSTEM ARCHITECTURE

4.1 High-Level Architecture

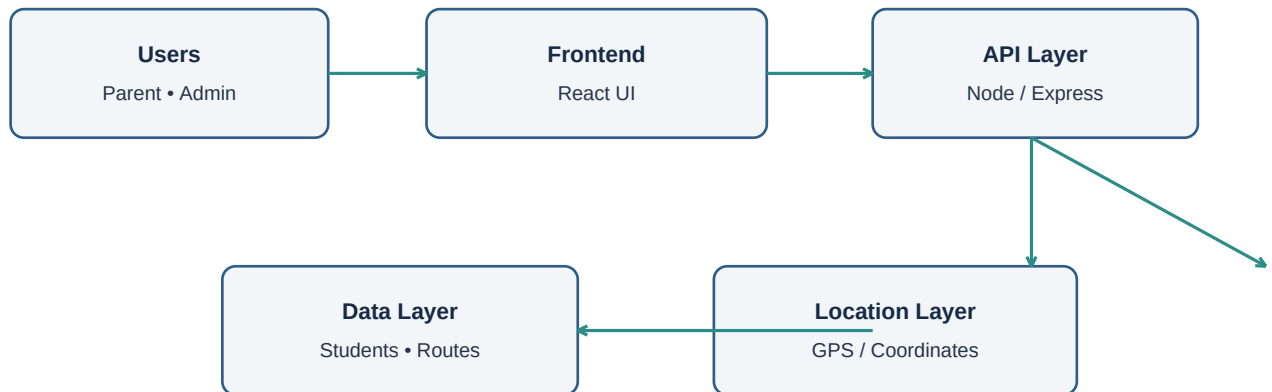


Figure 4.1 — High-level architecture of the School Bus Tracking Portal

4.2 Description of Each Layer

Presentation Layer: contains the login page, parent dashboard, tracking interface, route information, alerts, and administrator screens.

Application/API Layer: receives requests, validates data, applies business rules and returns structured responses. It also acts as the boundary between the user interface and persistent storage.

Data Layer: stores structured entities and event records such as students, buses, routes, stops, trips and attendance events.

Location Layer: supplies or stores vehicle coordinates and specific stop coordinates, allowing the interface to render location-aware information.

4.3 Request / Response Flow

A typical parent tracking request follows the sequence: authenticated parent → frontend tracking page → API request → route/student/trip lookup → latest location query → JSON response → map and stop-list update. When a new GPS event is received, the same information flow can be used to refresh the displayed bus position.

4.4 Deployment Architecture

For development, the frontend, API server and database may run locally. In a production environment, the frontend can be hosted on a static web platform, the Node/Express API can run on a managed server, and the database can use a managed MySQL service. HTTPS, restricted database access, environment variables and logging should be applied before public deployment.

4.5 Security Boundary

Authentication and authorisation should be enforced at the API boundary rather than relying only on frontend route protection. A parent should receive only the records associated with the authenticated account and child, while administrative access should be restricted to authorised staff.

CHAPTER 5

SYSTEM DESIGN

5.1 Design Principles

- Safety-first information hierarchy.
- Minimum necessary exposure of student data.
- Clear distinction between current location and scheduled stop.
- Consistent role-based navigation.
- Responsive and accessible user interface.
- Modular data entities to support future backend integration.

5.2 ER Diagram / Database Schema

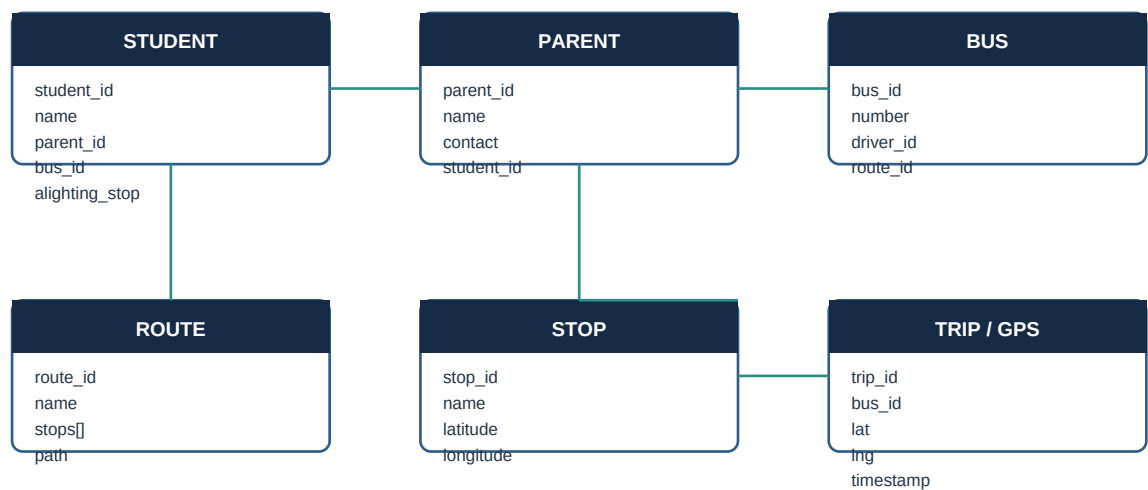


Figure 5.1 — Conceptual entity relationship model

5.3 Core Entities

Entity	Purpose
Student	Stores child/student transport identity and associations.
Parent	Stores authorised parent account and child relationship.
Bus	Represents a school bus and its operational identifiers.
Driver	Stores driver/transport staff information.
Route	Defines an ordered journey and its stops.
Stop	Stores stop name, specific location and coordinates.
Trip	Represents a particular bus journey on a route.

Boarding	Records a student's boarding event.
Alighting	Records a student's get-down event.
GPS Update	Stores bus coordinates and timestamp.
Alert	Stores safety/transport notifications.

5.4 Location-Specific Stop Design

Each stop should contain a stable identifier, a user-friendly name, geographic coordinates, and an ordered position within the route. For example, a stop record can conceptually contain `stop_id`, `stop_name`, `address/location`, `latitude`, `longitude` and `sequence`. This avoids treating all stops as generic labels and enables the tracking screen to communicate the actual location relevant to the child.

5.5 Child Alighting Stop Logic

The child's scheduled alighting stop is derived from the student's transport assignment and the route's stop list. The frontend can compare the student's `alighting_stop_id` with each rendered stop. The matching stop receives a distinct visual state and a label such as "Child's Stop" or "Get Down Stop".

CHAPTER 6

IMPLEMENTATION

6.1 Module-Wise Implementation

Module	Implementation Responsibility
Authentication	Login, role selection/authorisation and protected navigation.
Parent Dashboard	Student summary, bus/trip status, route and alerts.
Tracking	Current/last-known bus location, route and stop markers.
Stop Highlighting	Matches child alighting stop to route stop and highlights it.
Admin Dashboard	Operational overview and management navigation.
Students	Student records and transport associations.
Buses / Drivers	Fleet and driver information.
Trips	Trip status, route and schedule information.
Boarding / Alighting	Event records for student movement.
GPS	Vehicle coordinate records and monitoring.
Alerts	Safety and transport notifications.

6.2 Frontend Logic

The frontend renders data into focused components. Login state determines the dashboard route, while transport data determines the cards, route list, map markers and alerts. Responsive CSS is used so that the login page opens with the form centered in the viewport on normal screens, avoiding unnecessary vertical scrolling.

6.3 API Integration

API integration is designed around resource-oriented endpoints. Example resources include /students, /buses, /routes, /stops, /trips, /boarding, /alighting, /gps and /alerts. The frontend can request JSON data and transform it into UI state. During prototype development, mock data can represent the same schema before the API is fully connected.

6.4 Authentication & Authorization

Authentication establishes who is using the portal. Authorisation determines what that user may access. Parent accounts should be restricted to their associated child's transport information. Administrator accounts can access operational modules. Credentials and tokens must never be hard-coded into production builds.

6.5 Error Handling

- Show clear user-facing messages when a request fails.
- Distinguish unavailable GPS from a valid stationary position.
- Handle missing student/route associations gracefully.
- Validate required fields before storing transport events.

- Log server-side failures without exposing sensitive information to users.

6.6 Responsive Login Alignment

The login page is designed around a viewport-aware layout. On normal desktop and laptop screens, the authentication card should be horizontally and vertically centred, with enough breathing room around it. On smaller screens, the layout should allow natural scrolling rather than forcing content outside the viewport.

CHAPTER 7

FEATURES

7.1 Role-Based Login

Provides separate entry paths for authorised parent and administrator users.

7.2 Parent Dashboard

Presents child-related transport information in a focused view.

7.3 Location-Specific Tracking

Uses meaningful stop information and geographic coordinates rather than relying only on generic stop labels.

7.4 Highlighted Child Stop

Marks the child's scheduled get-down location with a distinct visual treatment.

7.5 Route Visualization

Displays ordered stops and, when location data is available, the route/bus position.

7.6 Boarding & Alighting

Maintains records that indicate when and where a student entered or left the bus.

7.7 Admin Transport Management

Provides navigation for students, buses, drivers, trips, GPS and alerts.

7.8 Alerts

Creates a channel for communicating transport or safety-related events.

7.9 Responsive Interface

Adapts dashboards and login layouts to different screen sizes.

7.10 Extensible Data Model

Supports future integration with live GPS devices, RFID/NFC readers and notification services.

7.11 Typical Parent Journey

- Open the portal and sign in.
- View the associated child and current transport information.
- Open the tracking view.
- Identify the bus's latest known location.
- Review the ordered stop list.
- Locate the visually highlighted child's get-down stop.
- Check alerts or trip status.

- Use the displayed location information to understand the journey without repeated manual updates.

7.12 Typical Administrator Journey

- Sign in using an authorised administrator account.
- Review dashboard activity.
- Open student, bus, driver or trip records.
- Monitor boarding/alighting events.
- Review GPS information and route status.
- Inspect alerts and operational activity.
- Use records for transport accountability and issue investigation.

CHAPTER 8

TESTING

8.1 Testing Strategy

Testing focuses on whether the portal behaves correctly from both user and system perspectives. Functional testing verifies page behaviour and workflows. API testing verifies request methods, validation and response structures when the backend is connected. Integration testing verifies that the frontend correctly consumes backend data.

8.2 Functional Test Cases

ID	Test Case	Expected Result	Acceptance Condition
TC-01	Valid parent login	Dashboard opens for authorised parent	Pass condition: correct parent view
TC-02	Invalid login	Access is rejected	Pass condition: error shown; dashboard not opened
TC-03	Student association	Correct child is displayed	Pass condition: associated student only
TC-04	Specific stop	Stop shows meaningful location	Pass condition: name/location/coordinates available
TC-05	Child stop highlight	Assigned alighting stop is highlighted	Pass condition: one relevant stop highlighted
TC-06	GPS update	Latest position appears	Pass condition: current/last-known marker updates
TC-07	Boarding event	Boarding record appears	Pass condition: correct student/trip/time
TC-08	Alighting event	Alighting record appears	Pass condition: correct stop/time
TC-09	Admin access	Admin modules are available	Pass condition: authorised admin view
TC-10	Responsive login	Login is centred	Pass condition: no unnecessary initial scrolling

8.3 Postman / API Testing

When REST endpoints are available, Postman can be used to test GET, POST, PATCH/PUT and DELETE operations. Requests should cover valid payloads, missing fields, invalid identifiers, duplicate records, and unauthorised access. Response codes and JSON structures should be recorded as evidence.

8.4 Location Testing

- Verify latitude and longitude are within valid geographic ranges.
- Verify a stop's coordinates remain associated with the correct stop ID.
- Verify the current bus marker is not mistaken for a scheduled stop.
- Verify stale GPS timestamps are distinguishable from current data.

- Verify the child's highlighted stop remains correct when route order changes.

8.5 Security Testing

Security testing should include authentication failures, access to another student's record, expired/invalid tokens, malformed request payloads and attempts to access administrator resources as a parent. The goal is to ensure that location information is exposed only to authorised users.

CHAPTER 9

DEPLOYMENT

9.1 Development Deployment

During development, the frontend can be started using the project's npm scripts and accessed through a local browser. If a backend is enabled, it should run as a separate local process with a configured API base URL. The database should run locally or through an approved development service.

9.2 Environment Configuration

Environment variables should be used for values that change between environments, such as API base URLs, database credentials, authentication secrets and external map-service keys. Sensitive values should not be committed to source control.

9.3 Hosting of Frontend and Backend

A production deployment can host the React frontend on a static hosting platform and the Node/Express backend on a managed application service. A managed relational database can host the transport records. HTTPS should be enforced for all public traffic, and the database should not be directly exposed to the public internet.

9.4 Operational Monitoring

Monitoring should track server availability, API errors, response times, database connectivity and GPS update freshness. Operational alerts should distinguish an application failure from an actual transport event so that technical problems do not create misleading safety notifications.

9.5 Backup and Recovery

Student, parent, route and transport records should be backed up regularly. Recovery procedures should be documented and periodically tested. GPS history may require a retention policy because location records can grow rapidly over time.

CHAPTER 10

CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

The project combines several domains that are individually manageable but become more complex when connected: authentication, student relationships, route ordering, boarding/alighting events, map coordinates, administrative workflows and live location. Maintaining consistency across these modules is a major development challenge.

Another challenge is presenting enough transport information to be useful without overwhelming the parent. A parent generally needs the child's current journey, bus position, relevant stop and alerts rather than a full administrative dataset.

10.2 Solutions Applied

- Separate parent and administrator workflows.
- Use structured entities for students, buses, routes, stops and trips.
- Use a dedicated alighting_stop relationship to identify the child's get-down point.
- Represent stops using coordinates and meaningful names.
- Keep frontend components modular so backend integration can be introduced progressively.
- Use responsive layout rules for login and dashboard screens.

10.3 Current Limitations

- Live GPS accuracy depends on the availability and quality of the location source.
- A browser-based prototype may use mock or last-known data until a real GPS feed is connected.
- The system requires careful privacy and access controls before handling real student locations.
- Map services may require API keys, quotas or internet access.
- Automated notifications require an external messaging service or school notification infrastructure.
- Production deployment requires stronger authentication, logging, backups and monitoring than a classroom prototype.

10.4 Privacy and Safety Considerations

Student location is sensitive information. A production system should minimise data exposure, enforce strong authentication, encrypt traffic, restrict administrator privileges, maintain audit logs and define data retention rules. Location history should not be displayed to unrelated users, and test datasets should avoid unnecessary real-world personal information.

CHAPTER 11

FUTURE ENHANCEMENTS

11.1 Planned Features

- True real-time GPS streaming from bus devices.
- Push notifications for boarding, alighting, arrival and delays.
- Geofencing around school, stops and predefined safety zones.
- ETA estimation based on current location and route progress.
- Driver-side trip application for controlled status updates.
- Emergency SOS workflow for authorised transport staff.
- Parent notification preferences and escalation settings.

11.2 Possible Hardware Integration

For younger children who may not carry smartphones, the system can be extended with RFID or NFC-based identification. A card, tag or wearable can be associated with a student ID. A reader installed at the bus door can record boarding and alighting events and send them to the backend. Such hardware should be implemented with secure device authentication and clear operational procedures.

11.3 Advanced Location Intelligence

Future versions can calculate estimated arrival times, detect route deviations, identify prolonged stops and compare actual journeys with expected routes. These features should be introduced carefully because false positives in a safety system can reduce trust.

11.4 Analytics

Administrative analytics could summarise route punctuality, recurring delays, boarding patterns, bus utilisation and alert frequency. Analytics should use aggregated information where possible and follow the same privacy principles as the main portal.

11.5 Possible Integrations

- School ERP/student information systems.
- SMS, email or push notification providers.
- Map and routing APIs.
- RFID/NFC readers.
- GPS trackers or telematics devices.
- Cloud monitoring and backup services.

CHAPTER 12

CONCLUSION

12.1 Summary of the Project

The School Bus Tracking Portal is a practical web-based approach to improving visibility and accountability in student transportation. It brings together parent access, administrator monitoring, student records, bus and route information, boarding and alighting events, GPS data and alerts into one structured system.

A central design decision is the use of location-specific stop information. Rather than showing only generalized stop labels, the system can associate each stop with a meaningful location and geographic coordinates. The child's scheduled get-down stop is then highlighted, helping the parent quickly identify the location that matters most to the child's journey.

12.2 What Was Achieved

- Defined a complete school-bus tracking portal architecture.
- Established parent and administrator workflows.
- Modelled students, buses, drivers, routes, stops and trip events.
- Designed location-aware tracking and child-stop highlighting.
- Defined functional and non-functional requirements.
- Established a testing strategy for frontend, API and safety-related workflows.
- Identified a practical roadmap for GPS, RFID/NFC and notification integration.

12.3 Skills Learned

- React-based component design and responsive UI development.
- REST API and backend architecture concepts.
- Database and entity-relationship modelling.
- Location and GPS data representation.
- Authentication and role-based access concepts.
- Functional and API testing.
- Team collaboration, debugging and technical documentation.

12.4 Final Statement

The project demonstrates how a focused software solution can address an everyday safety problem through structured data, clear interfaces and location-aware functionality. With production-grade backend, authentication, GPS hardware and privacy controls added, the prototype can evolve into a more complete school transport management platform.

CHAPTER 13

REFERENCES

The following resources provide technical background and implementation guidance for the technologies and methods discussed in this report.

- React documentation — component-based user interface development.
- Node.js documentation — server-side JavaScript runtime.
- Express.js documentation — routing and REST API development.
- MySQL documentation — relational database concepts and administration.
- MDN Web Docs — HTML, CSS, JavaScript and browser APIs.
- Postman documentation — API request testing and collections.
- General geolocation and mapping API documentation used during implementation, where applicable.
- Course notes and faculty-provided project guidance.

Note: Specific external API providers and deployment services should be listed in the final project repository if they are used in the deployed version.

CHAPTER 14

APPENDICES

14.1 Suggested Application Screenshots

Ref	Screenshot	Purpose
A1	Login Page	Parent/Admin login screen showing the centred authentication form.
A2	Parent Dashboard	Child information, bus status and transport summary.
A3	Live Tracking	Map with current/last-known bus location and route.
A4	Highlighted Stop	Route list/map showing the child's get-down stop clearly highlighted.
A5	Admin Dashboard	Administrative transport overview.
A6	Students	Student records and transport associations.
A7	Buses / Drivers	Fleet and driver records.
A8	Trips	Trip and route monitoring.
A9	Boarding / Alighting	Student movement event records.
A10	GPS / Alerts	Location monitoring and safety alerts.

14.2 Sample API Resource Map

Resource	Typical Purpose
/students	Student records
/parents	Parent/account association
/buses	Bus records
/drivers	Driver records
/routes	Route definitions
/stops	Location-specific stops
/trips	Trip instances
/boarding	Boarding events
/alighting	Alighting events
/gps	Vehicle location updates
/alerts	Transport/safety alerts

14.3 Sample Stop Data

A stop record should contain enough information to uniquely identify the location and maintain its order within the route. The coordinates above are illustrative placeholders, not a real student's location.

```
{
  "stopId": "STOP_07",
  "stopName": "Child's Designated Stop",
  "location": "Specific locality / landmark",
  "latitude": 17.000000,
  "longitude": 78.000000,
  "sequence": 7
}
```

14.4 Sample Student Transport Relationship

A student record can reference the assigned bus and route as well as the student's designated alighting stop. The frontend uses this relationship to highlight the correct stop without requiring the parent to interpret the entire route manually.

14.5 User Manual — Parent

- Open the portal and enter authorised credentials.
- After login, verify the displayed child.
- Open the tracking view to see the route and current/last-known bus position.
- Review the location-specific stop list.
- Look for the highlighted “Child's Stop” marker.
- Review any alerts shown on the dashboard.
- Contact the school transport office if information appears stale or inconsistent.

14.6 User Manual — Administrator

- Sign in using authorised administrator credentials.
- Review the dashboard for active transport information.
- Use Students, Buses, Drivers and Trips modules to inspect records.
- Use Boarding and Alighting modules to review movement events.
- Use GPS to inspect vehicle location data.
- Review Alerts for safety and operational events.
- Record or escalate anomalies according to school transport procedures.

END OF PROJECT DOCUMENTATION